

Specular Grassmann-Polarity Index Method

USER MANUAL

Carlos Polanco

August 2026

CONTENTS

Contents	i
I Understanding SGPMAIN (10.0 versión): The Program's Vision and Architecture	1
1 Introduction: Why SGPMAIN (10.0 versión) Exists	3
1.1 The Fundamental Problem	3
1.2 The Four Pillars of SGPMAIN (10.0 versión)	3
1.3 The Three-Level Architecture	5
1.4 The Role of config.json in the Architecture	7
1.5 What SGPMAIN (10.0 versión) Does Not Do	9
1.6 The User's Journey Through This Manual	9
2 The Complete Workflow: From Sequence to Peptide	11
2.1 Overview of the Workflow	11
2.2 Stage 1: Data Loading	12

2.3	Stage 2: PIM Computation	12
2.4	Stage 3: Group Statistics	13
2.5	Stage 4: Comparison and Similarity	13
2.6	Stage 5: Chemical Profiling	14
2.7	Stage 6: PIDP Analysis	15
2.8	Stage 7: Therapeutic Profiling	15
2.9	Stage 8: Peptide Design Engine	16
2.10	Stage 9: Drug-Likeness Filtering	17
2.11	The Role of config.json in the Workflow	17
 II Mathematical Foundations and Metrics		 19
3	The Polarity Index Method (PIM): Mathematical Foundations	21
3.1	Introduction to the Mathematical Framework	21
3.2	The 16 Polarity Interactions	23
3.3	Biological Weights	23
3.4	Metric Signature	23
3.5	Functions for PIM Computation	24
4	Grassmann Algebra: Fundamental Operations	25
4.1	Introduction to Grassmann Algebra	25
4.2	Wedge Product and Corrected Similarity	25
4.3	Specular Reflection	26
4.4	Clifford Signature	27
5	The 18 Mathematical Metrics	29
5.1	Shannon Entropy	29
5.2	Jensen-Shannon Divergence	29
5.3	Gini Coefficient	30
5.4	Structural Complexity Index	30
5.5	Functional Modularity Index	31
5.6	Moran's I	32
5.7	Hellinger Distance	32
5.8	Spearman Correlation	33
5.9	Grassmann Projection Distance	33
5.10	Fubini-Study Metric	33
5.11	Ricci Curvature	34
5.12	Wasserstein Distance	34
5.13	Fractal Dimension	35
5.14	Radon Transform	35
5.15	Karhunen-Loève Decomposition	36
5.16	Polarity Laplacian	36
5.17	Commutator Norm	37
5.18	Anticommutator Similarity	37

6	How Metrics Interact with config.json	39
6.1	The Role of Metric Weights	39
6.2	The Role of Target-Specific Ranges	40
6.3	How the Validation Works	40
6.4	Why This Matters for Peptide Design	41
III	Case Study: West Nile Virus	43
7	Application to West Nile Virus	45
7.1	Introduction to West Nile Virus	45
7.2	NILE1: Small Envelope Protein M	45
7.3	NILE2: Envelope Glycoprotein E	46
7.4	PIM Profiles for WNV Proteins	46
7.5	Similarity Analysis: NILE1 vs NILE2	47
7.6	WNV Chemical Profile Summary	47
7.7	Hotspot Analysis	47
7.8	PIDP Analysis for WNV Proteins	48
7.9	Peptide Design for WNV	48
A	Installation and System Requirements	51
A.1	System Requirements	51
A.2	Required Packages	51
A.3	Installation Steps	51
A.4	Running SGPMAN (10.0 versión)	52
A.5	Quick Reference	52
	Bibliography	53

Abstract

This document presents the complete user’s manual and conceptual framework of SGPMAN (10.0 versión) (Specular Grassmann-Polarity Index Method), a computational platform for protein analysis and peptide design. The system is built on four fundamental pillars: (1) the Polarity Index Method (PIM) as a representation of proteins as 16-dimensional vectors, (2) Grassmann Algebra as the mathematical engine for comparing and transforming polarity profiles, (3) Transfer Learning with ESM2 embeddings for enriched sequence representation, and (4) Multi-Objective Bayesian Optimization for peptide design.

New Features in SGPMAN (10.0 versión):

- **Hybrid Configuration with `config.json`:** External configuration file with structured fallback to hardcoded values. This enables target-specific metric ranges, dynamic metric weights, and virus-specific parameters without modifying the source code.
- **Target-Specific Ranges:** Each virus or protein target now has its own set of metric ranges (PIM similarity, entropy, Grassmann distance, etc.), validated against the designed peptide.
- **Dynamic Metric Weights:** The weights of all 13 metrics (PIM, entropy, Grassmann, Hodge, etc.) can be adjusted per target via `config.json`.
- **Pathogen-Specific Inhibitors:** Known inhibitors for each virus are loaded from `config.json`, enabling context-aware comparison.
- **Processing Parameters:** Batch size, sample size, bootstrap iterations, and worker count are now configurable without code changes.

Existing Features Maintained from v9.0:

- Transfer Learning with ESM2 (esm2_t6_8M_UR50D, 320-dimensional embeddings)
- Multi-Objective Predictor (antiviral activity, cytotoxicity, stability, selectivity index)
- Peptide Generator with Bayesian Optimization (Gaussian Process + UCB acquisition)
- Drug-Likeness Filters (Lipinski-modified, Veber rules, toxicity, stability, synthesis)
- Pareto Front Optimization and Active Learning
- LoRA Fine-Tuning (rank $r = 8$, alpha $\alpha = 16$, dropout $p = 0.1$)
- Grassmann Multilevel (k=1,2,3) with SVD caching
- ChemicalProfiler with 54 unified properties
- **PIDP** (metapredict + AIUPred) - **See Section on PIDP Analysis for detailed interpretation**
- TherapeuticProfiler (synthetic peptide design)

- All 18 mathematical metrics (see Table 5)

All mathematical formulas are included with clear explanations. Flow diagrams, property tables, and concrete examples using the West Nile virus (WNV) envelope proteins (NILE1 and NILE2) as a case study are included.

Version Information: This manual corresponds to SGPMAN (10.0 versión) (file: SGPMAN10.0v.py).

Part I

Understanding SGPMAN (10.0 versión): The Program's Vision and Architecture

INTRODUCTION: WHY SGPMAN (10.0 VERSION) EXISTS

1.1 The Fundamental Problem

Proteins are the molecular machines of life. They catalyze reactions, transport molecules, recognize pathogens, and transmit signals. Understanding how proteins work—and how to design molecules that interact with them—is one of the most important challenges in modern biology and medicine.

For decades, the dominant approach to understanding proteins has been through sequence analysis. We compare amino acid sequences, look for conserved motifs, and infer function from homology. This approach has been enormously successful, but it has a fundamental limitation: it treats proteins as strings of letters rather than as physical entities with specific physicochemical properties.

The insight behind SGPMAN (10.0 version) is that protein function is not determined by the exact sequence of amino acids, but by the spatial and temporal organization of polarity transitions along the polypeptide chain. Two proteins with completely different sequences can have very similar functions if they share similar polarity patterns. Conversely, proteins with similar sequences can have different functions if their polarity patterns diverge.

This insight is not just a theoretical curiosity—it has profound practical implications. It means that we can design therapeutic peptides not by copying natural sequences, but by understanding the polarity patterns that underlie protein function. It means we can identify new drug targets by finding proteins that share polarity patterns with known targets. And it means we can move beyond the limitations of sequence alignment to a more fundamental understanding of protein function.

1.2 The Four Pillars of SGPMAN (10.0 version)

SGPMAN (10.0 version) is built on four pillars that work together to transform raw sequence data into actionable biochemical insights:

Pillar 1: The Polarity Index Method (PIM)

The Polarity Index Method is the foundational representation layer. It transforms the complex, high-dimensional space of amino acid sequences into a compact 16-dimensional vector space. Each dimension corresponds to a specific polarity transition: from positive to negative charge, from polar to nonpolar, and so on.

This transformation is mathematically rigorous yet biologically interpretable. Each transition

directly corresponds to a specific physicochemical interaction that can be measured and validated experimentally. The resulting 16-dimensional vector is a fingerprint of the protein's polarity organization that is independent of sequence length and robust to minor sequence variations.

Pillar 2: Grassmann Algebra

Grassmann Algebra is the mathematical engine that operates on the PIM vectors. It provides a natural framework for comparing, transforming, and analyzing vector subspaces. The key insight is that the wedge product of two PIM vectors gives a measure of their geometric relationship that is both mathematically sound and biologically meaningful.

The corrected wedge similarity, introduced in version 9.0, is a more stringent measure than previous versions. A value of 0.12 indicates near-orthogonality of the polarity profiles, representing a substantial functional difference between the compared proteins. This improved discriminative power is essential for distinguishing between subtly different protein functions.

Pillar 3: Transfer Learning with ESM2

Protein language models like ESM2 have revolutionized computational biology. Trained on millions of protein sequences, these models learn representations that capture evolutionary, structural, and functional information. The embedding of a protein sequence from ESM2 is a 320-dimensional vector that encodes much more information than the sequence alone.

In SGPMAN (10.0 versión), ESM2 embeddings are combined with PIM vectors and physicochemical properties to create enriched feature vectors of 413 dimensions. This combination captures both the physicochemical essence of the protein (through PIM) and its evolutionary context (through ESM2).

Pillar 4: Hybrid Configuration with `config.json`

Version 9.1 introduces a new pillar: hybrid configuration. The **`config.json`** file provides external, structured configuration that enables target-specific optimization without modifying the source code.

The Fallback Mechanism

One of the key design principles of SGPMAN (10.0 versión) is robustness. The program should always run, even if the **`config.json`** file is missing or invalid. To achieve this, the system uses a fallback mechanism:

1. The program checks for the existence of **`config.json`** in the working directory.
2. If the file exists, it attempts to load and validate it.
3. If the file is valid, the program uses the values from **`config.json`**.

4. If the file is missing or invalid, the program falls back to hardcoded default values.

This ensures that SGPMAN (10.0 versión) always runs, regardless of configuration issues. The user is informed of which source is being used (**config.json** or default), so they know exactly what parameters are in effect.

Target-Specific Ranges

Each virus or protein target now has its own set of metric ranges. For West Nile virus, the PIM similarity range might be 0.72-0.90, while for Ebola it might be 0.80-0.95. These ranges are used to validate designed peptides and to guide the optimization process.

Dynamic Metric Weights

The 13 metrics used in peptide design (PIM, entropy, Grassmann distance, Hodge complementarity, etc.) now have weights that can be adjusted per target. This allows the system to emphasize different aspects of the polarity profile depending on the specific pathogen.

1.3 The Three-Level Architecture

SGPMAN (10.0 versión) operates through a carefully designed three-level architecture that transforms raw sequence data into actionable biochemical insights. Each level builds upon the previous one, creating a coherent pipeline from sequence to function.

Level 1: PIM - The Foundation

At the most fundamental level, SGPMAN (10.0 versión) implements the Polarity Index Method. This level takes as input the amino acid sequence of a protein and produces as output a 16-dimensional vector. The transformation is accomplished through the following steps:

First, each amino acid in the sequence is classified into one of four polarity categories: positively charged (P^+ : H, K, R), negatively charged (P^- : D, E), neutral polar (N: C, G, N, Q, S, T, Y), and nonpolar hydrophobic (NP: A, F, I, L, M, P, V, W). This classification captures the essential electrostatic and hydrogen-bonding characteristics of each residue.

Second, the sequence is scanned to count the frequency of each possible transition between polarity categories. There are 16 possible transitions (four categories to four categories), and these counts form the components of the PIM vector.

Third, the vector is normalized so that all components sum to one, producing a probability distribution that is independent of sequence length. This normalization ensures that proteins of different lengths can be compared directly.

The result of Level 1 is a compact, length-independent representation of the protein's polarity organization that captures the essential physicochemical information needed for functional analysis. This representation is the foundation upon which all subsequent analyses are built.

Level 2: Grassmann Algebra and Transfer Learning - The Analytical Engine

At the intermediate level, SGPMAN (10.0 versión) applies Grassmann Algebra and Transfer Learning to operate on the PIM vectors produced by Level 1. Grassmann algebra provides a natural mathematical framework for working with vector subspaces and their geometric relationships. ESM2 provides enriched evolutionary and functional context.

The key operations at this level include:

The Wedge Product: This computes the area of the parallelogram spanned by two PIM vectors, providing a measure of their geometric relationship that captures both similarity and orthogonality.

Specular Reflection: This tests for charge-inverted functional equivalence by inverting the charge components of the PIM vector. This is a novel concept that captures a subtle form of functional similarity.

The Clifford Signature: This combines multiple geometric and information-theoretic measures into a comprehensive fingerprint of the protein's polarity profile.

ESM2 Embedding Extraction: This captures evolutionary, structural, and functional information from protein language models, enriching the representation.

LoRA Fine-Tuning: This enables efficient adaptation of ESM2 to domain-specific tasks with minimal parameters (rank $r = 8$, alpha $\alpha = 16$, dropout $p = 0.1$).

The result of Level 2 is a set of geometric measures and enriched features that describe the relationships between proteins in terms of their polarity organization and evolutionary context.

Level 3: Derived Properties, PIDP, and Peptide Design - The Biochemical Outputs

At the highest level, SGPMAN (10.0 versión) transforms the abstract vector space of PIM vectors into concrete biochemical properties that are directly interpretable by experimental scientists. This level comprises four complementary modules:

Chemical Properties Module: This module derives over 54 physicochemical properties from the PIM vector, including net charge, isoelectric point (pI), GRAVY (hydrophobicity), solubility, conformational stability (ΔG), melting temperature (T_m), glycosylation sites, phosphorylation sites, drug-binding hot spots, reactivity profile, metal binding sites, membrane permeability, lipid binding preferences, and buffer stability recommendations.

Protein Intrinsic Disorder Prediction (PIDP) Module: This module provides critical structural insights using two independent methods: metapredict (machine learning-based) and AIUPred (biophysical model-based). Both methods produce per-residue disorder scores and percentages of disordered residues at multiple thresholds. The PIDP analysis is particularly important because it reveals

whether a protein has flexible regions that may be important for function or immune evasion. See Section on PIDP Analysis for detailed interpretation.

Multi-Objective Predictor Module: This module predicts multiple properties of antiviral peptides: antiviral activity (IC50/EC50), cytotoxicity (CC50), stability (plasma half-life), selectivity index (CC50/IC50), hemolytic activity, and immunogenicity.

Peptide Design Module: This module designs optimized peptides targeting specific proteins. It identifies critical polarity transitions, designs peptide sequences that mimic these transitions, uses Gaussian Process optimization with UCB acquisition, maintains Pareto front of non-dominated solutions, evaluates drug-likeness using multiple filters, and generates synthesis and formulation recommendations.

1.4 The Role of *config.json* in the Architecture

The ***config.json*** file is a new addition in version 9.1 that fundamentally changes how SGPMAN operates. It sits at the boundary between Level 2 and Level 3, providing target-specific parameters that guide both the analytical engine and the biochemical outputs.

Why *config.json* Exists:

In previous versions, all parameters were hardcoded. To change the metric weights, you had to modify the source code. To add a new virus, you had to edit the MAIN_GROUP list. To adjust the batch size, you had to find the right variable in the code and change it.

This approach worked, but it had significant limitations. It made the program inflexible, hard to maintain, and difficult to adapt to new targets. It also made it impossible to have different settings for different viruses without maintaining multiple versions of the code.

How *config.json* Works:

The ***config.json*** file is a structured JSON document that contains all the configurable parameters of SGPMAN (10.0 version). It is divided into sections that mirror the architecture of the program:

- **Metadata:** Version information, generation date, and source information.
- **Metric Weights:** The weights of the 13 metrics used in peptide design.
- **Target Ranges:** For each virus or protein target, the acceptable ranges for each metric (PIM similarity, entropy, Grassmann distance, etc.).
- **Classification Thresholds:** The thresholds for excellent, good, moderate, and poor drug candidates.
- **Known Inhibitors:** For each virus, the known inhibitors with their IC50 and Ki values.
- **Virus-Specific Parameters:** UniProt IDs, PDB structures, domains, key residues, serotypes, and clinical considerations.

- **Peptide Design Parameters:** Preferred amino acids, length ranges, secondary structure preferences, and known resistance mutations.
- **Processing Parameters:** Batch size, sample size, bootstrap iterations, and worker count.
- **Data Paths:** Paths to external files like `chembl_uniprot.txt` and `apd_natural.fasta`.

Target-Specific Optimization:

The most powerful feature of **config.json** is target-specific optimization. Each virus or protein target can have its own set of parameters, allowing the system to be optimized for the specific characteristics of that target.

For example, West Nile virus (WNV) has a moderately complex envelope protein (E) with a specific fusion loop. The **config.json** for WNV might have:

- PIM similarity range: 0.72-0.90 (moderately strict)
- Entropy range: 2.5-3.6 (moderate diversity)
- Grassmann distance range: 0.30-0.55 (moderate distance)
- Metric weights: higher weight on PIM (0.28) and Grassmann (0.14)
- Known inhibitors: JNJ-1802, Compound_816, AR-23, RV-23
- Key residues: D114, E115, Y116 (fusion loop)
- Preferred amino acids: K, R, D, E, L, V

For Ebola virus, which has a more complex glycoprotein (GP), the **config.json** might have:

- PIM similarity range: 0.80-0.95 (very strict)
- Entropy range: 3.0-4.0 (high diversity)
- Grassmann distance range: 0.20-0.40 (close distance)
- Metric weights: higher weight on Hodge complementarity (0.10) and curvature (0.10)
- Known inhibitors: ZMapp, REGN-EB3, mAb114
- Key residues: D522, E523, Y524 (fusion loop)
- Preferred amino acids: K, R, L, V, I, F, W

This target-specific optimization is what makes SGPMAN (10.0 version) so powerful. Instead of a one-size-fits-all approach, the system can be fine-tuned for each specific pathogen.

1.5 What SGPMAN (10.0 versión) Does Not Do

Understanding what SGPMAN (10.0 versión) does not do is as important as understanding what it does.

SGPMAN (10.0 versión) is not a structure prediction tool. It does not predict the three-dimensional structure of proteins. Instead, it predicts the polarity organization of proteins, which is a different (and complementary) type of information.

SGPMAN (10.0 versión) is not a docking tool. It does not predict how two proteins will bind to each other. Instead, it predicts whether two proteins are likely to share similar functions based on their polarity profiles.

SGPMAN (10.0 versión) is not a drug design tool. It does not design drugs in the traditional sense. Instead, it designs peptides that mimic the polarity patterns of target proteins, which can then be synthesized and tested as potential therapeutics.

SGPMAN (10.0 versión) is not a substitute for experiments. The predictions and designs produced by the system are hypotheses that need to be validated experimentally. The system provides the basis for designing experiments, not the results of those experiments.

1.6 The User's Journey Through This Manual

This manual is organized to guide you through the SGPMAN (10.0 versión) system from first principles to practical application:

Part I: Understanding SGPMAN (10.0 versión) (this part) provides the conceptual foundation. It explains why the system exists, what it does, and how it is structured.

Part II: Mathematical Foundations provides the detailed mathematics behind each component. It includes definitions, formulas, and examples for each metric.

Part III: Case Studies shows the system in action, using West Nile virus as a concrete example.

Appendix: Installation and System Requirements provides practical information on how to install and run the system.

We recommend reading Part I in its entirety before proceeding to the mathematical details. The conceptual foundation will help you understand why the mathematics is structured the way it is.

THE COMPLETE WORKFLOW: FROM SEQUENCE TO PEPTIDE

This chapter describes the complete workflow of SGPMAN (10.0 versión), from loading protein sequences to designing therapeutic peptides. The workflow is a pipeline that transforms raw data into actionable insights.

2.1 Overview of the Workflow

The SGPMAN (10.0 versión) workflow consists of several stages, each building on the previous one:

1. **Data Loading:** The system reads protein sequences from FASTA files, one group at a time. Each group corresponds to a virus strain, protein family, or reference dataset.
2. **PIM Computation:** For each protein sequence, the system computes the 16-dimensional PIM vector. This is the foundational representation that underlies all subsequent analysis.
3. **Group Statistics:** For each group of proteins, the system computes statistics including the centroid (mean PIM vector), covariance matrix, and various geometric measures.
4. **Comparison and Similarity:** The system compares groups and individual proteins using the wedge similarity and other geometric measures.
5. **Chemical Profiling:** For each group, the system derives 54 chemical properties from the PIM vector.
6. **PIDP Analysis:** The system predicts intrinsic disorder for each target protein using metapredict and AIUPred. This analysis is critical for understanding protein flexibility, which has implications for immune evasion, receptor binding, and therapeutic targeting.
7. **Therapeutic Profiling:** The system designs therapeutic peptides targeting the selected protein, using all available metrics and `config.json` parameters.
8. **Peptide Design Engine:** The system uses Bayesian optimization to generate optimized peptide candidates.
9. **Drug-Likeness Filtering:** The system evaluates peptide candidates against multiple drug-likeness criteria.

2.2 Stage 1: Data Loading

The data loading stage is the entry point of the workflow. The system reads FASTA files from the working directory, each containing protein sequences for a specific group.

Input: FASTA files with the naming pattern `{group_name}.unico.dat0`.

Processing: The system streams each file, reading sequences one at a time. This streaming approach is essential for handling the large files (up to 73 GB) that the system is designed to process.

Output: For each group, the system stores a sample of up to 200 protein sequences (configurable via `config.json`). The sample is maintained using reservoir sampling, ensuring that it is representative of the full dataset.

Reservoir Sampling

The system uses reservoir sampling to maintain a representative subset of proteins from each group. This is essential for handling large files without running out of memory.

The algorithm is simple but powerful:

1. The first n proteins are stored directly in the sample.
2. For each subsequent protein, the system generates a random number between 0 and i (where i is the number of proteins seen so far).
3. If the random number is less than n , the system replaces a randomly selected protein in the sample with the new one.
4. This ensures that every protein in the dataset has an equal probability of being in the final sample.

2.3 Stage 2: PIM Computation

The PIM computation stage transforms each protein sequence into a 16-dimensional PIM vector.

Input: Protein sequence (as a string of amino acids).

Processing: The system classifies each amino acid into one of four polarity categories, counts the frequency of each polarity transition, and normalizes the resulting vector.

Output: A 16-dimensional PIM vector where each component is a probability (the sum of all components is 1).

Polarity Classification

The system uses the following polarity classification:

- **P⁺ (Positively Charged):** H, K, R
- **P⁻ (Negatively Charged):** D, E
- **N (Neutral Polar):** C, G, N, Q, S, T, Y
- **NP (Nonpolar):** A, F, I, L, M, P, V, W

This classification is based on the physicochemical properties of the amino acids and is consistent with standard biochemistry.

2.4 Stage 3: Group Statistics

The group statistics stage computes summary statistics for each group of proteins.

Input: PIM vectors for all proteins in a group.

Processing: The system computes the centroid (mean PIM vector), covariance matrix, standard deviation, and various geometric measures.

Output: A GroupStatistics object containing all the computed statistics.

The Centroid

The centroid of a group is the mean PIM vector of all proteins in the group. It represents the "average" polarity profile of the group.

Mathematical Definition:

$$\bar{v} = \frac{1}{n} \sum_{i=1}^n v_i \quad (2.1)$$

where n is the number of proteins in the group.

2.5 Stage 4: Comparison and Similarity

The comparison stage measures the similarity between groups and individual proteins.

Input: PIM vectors and group statistics.

Processing: The system computes the wedge similarity between the target and each group, as well as between individual proteins.

Output: Similarity scores and rankings.

The Wedge Similarity

The wedge similarity is the primary measure of similarity between two PIM vectors. It is computed using the corrected wedge product.

Mathematical Definition:

$$\text{WedgeSim}(v, w) = 1 - \frac{\|\omega_{\mathcal{K}}(v, w)\|_2}{\|v\|_2\|w\|_2} \quad (2.2)$$

The wedge similarity ranges from 0 to 1, where 1 indicates identical polarity profiles and 0 indicates orthogonal profiles.

2.6 Stage 5: Chemical Profiling

The chemical profiling stage derives chemical properties from the PIM vector.

Input: PIM vector for a protein or group.

Processing: The system applies a series of formulas to compute chemical properties.

Output: A comprehensive chemical profile with 54 properties.

Key Chemical Properties

The system computes over 20 chemical properties organized into 14 categories:

- Charge properties (net charge, charge density, charge balance)
- pKa/pI properties (isoelectric point)
- Electrostatic properties (surface charge density)
- Hydrophobicity properties (GRAVY score)
- Glycosylation sites (N-linked and O-linked)
- PTM sites (phosphorylation, acetylation, ubiquitination)
- Solubility properties (solubility, aggregation score)
- Stability properties (ΔG , T_m)
- Hotspots (drug-binding sites)
- Reactivity properties (cysteine, lysine, tyrosine accessibility)
- Metal binding properties (calcium, zinc)

- Membrane permeability properties
- Lipid binding properties
- Buffer stability properties

2.7 Stage 6: PIDP Analysis

The **PIDP analysis stage is a critical component of the workflow**. It predicts intrinsic disorder for each target protein using two independent methods: metapredict and AIUPred.

Input: Target protein sequences.

Processing: The system uses metapredict and AIUPred to predict per-residue disorder scores.

Output: Disorder percentages at multiple thresholds (0.3, 0.4, 0.5) and mean disorder scores.

Interpreting PIDP Results

The PIDP results provide crucial information about protein structure and function:

- **0-10% disorder:** Predominantly ordered protein. Suitable for crystallization. This indicates a well-defined structure with limited flexibility.
- **10-30% disorder:** Flexible regions present. May be important for function, such as molecular recognition or binding. These proteins have some structural plasticity.
- **30-50% disorder:** Significant disorder. Likely an intrinsically disordered protein (IDP) or has large intrinsically disordered regions (IDRs). These proteins are highly flexible and may be involved in signaling or regulation.
- **>50% disorder:** Highly disordered. Consider IDP-specific methods for analysis. These proteins lack stable tertiary structure and may function through conformational ensembles.

Why PIDP Matters: For viral proteins, disorder is often associated with immune evasion, receptor binding flexibility, and membrane fusion mechanisms. Understanding the disorder profile of a viral protein can inform therapeutic strategies and vaccine design.

2.8 Stage 7: Therapeutic Profiling

The therapeutic profiling stage designs therapeutic peptides targeting the selected protein.

Input: Target protein PIM vector and group statistics.

Processing: The system identifies critical polarity differences between the target and host proteins, and designs a peptide sequence that mimics the target’s polarity profile.

Output: A therapeutic peptide with predicted activity, physicochemical properties, and comparison with known inhibitors.

Peptide Design Strategy

The system uses a weighted combination of all 13 metrics to design the peptide:

$$\text{Composite} = \sum_{m \in \mathcal{M}} w_m \cdot \Delta_m \quad (2.3)$$

where $\mathcal{M}$ is the set of metrics, w_m are the metric weights (from `config.json`), and Δ_m are the differences between the target and host.

2.9 Stage 8: Peptide Design Engine

The peptide design engine uses Bayesian optimization to generate optimized peptide candidates.

Input: Target sequence, training data from APD.

Processing: The system uses Gaussian Process regression with UCB acquisition to explore the peptide sequence space.

Output: A Pareto front of optimized peptide candidates.

Bayesian Optimization

The system uses Gaussian Process regression to model the relationship between peptide sequence and desired properties:

1. Initialize with a set of candidate peptides.
2. Evaluate candidates using the multi-objective predictor.
3. Train the GP model on the evaluated candidates.
4. Use UCB acquisition to select the next candidate.
5. Mutate the selected candidate to generate new sequences.
6. Repeat steps 2-5 for N iterations.
7. Return the Pareto front of optimal candidates.

2.10 Stage 9: Drug-Likeness Filtering

The drug-likeness filtering stage evaluates peptide candidates against multiple criteria.

Input: Peptide candidates and their predicted properties.

Processing: The system applies Lipinski-modified rules, Veber rules, toxicity filters, stability filters, and synthesis feasibility filters.

Output: A drug-likeness score and recommendation for each candidate.

The Total Drug-Likeness Score

The total drug-likeness score is a weighted sum:

$$\mathcal{D}(S) = \sum_{f \in \mathcal{F}} w_f \cdot \text{Filter}_f(S) \quad (2.4)$$

where $\mathcal{F}$ is the set of filters and w_f are their weights.

2.11 The Role of config.json in the Workflow

The **config.json** file is central to the entire workflow. It influences the workflow at several key points:

1. **Data Loading:** The processing parameters (batch size, sample size, etc.) are read from **config.json**.
2. **Comparison and Similarity:** The target-specific ranges are used to validate the similarity scores.
3. **Therapeutic Profiling:** The metric weights from **config.json** are used in the peptide design.
4. **Peptide Design Engine:** The drug-likeness thresholds from **config.json** are used for candidate evaluation.
5. **All Stages:** The data paths (chembl_uniprot.txt, apd_natural.fasta) are read from **config.json**.

This integration ensures that the **config.json** file is not an afterthought but a core component of the workflow.

Part II

Mathematical Foundations and Metrics

THE POLARITY INDEX METHOD (PIM): MATHEMATICAL FOUNDATIONS

This chapter presents the mathematical foundations of the Polarity Index Method (PIM), including the definition of amino acid polarity, the construction of the PIM vector, the 16 polarity interactions, biological weights, the metric signature, and key functions for PIM computation.

3.1 Introduction to the Mathematical Framework

The Polarity Index Method (PIM) is built on a rigorous mathematical foundation that transforms the complex, discrete space of amino acid sequences into a continuous vector space. This transformation is essential because it enables the application of powerful mathematical tools—linear algebra, differential geometry, and information theory—to the analysis of protein sequences. The key insight is that the physicochemical properties of proteins can be captured by the frequencies of transitions between polarity categories, which are independent of the specific amino acid identities.

The mathematical framework has several important properties that make it suitable for protein analysis:

1. Well-posedness: The transformation is uniquely defined for any sequence of sufficient length.
2. Continuity: Small changes in the sequence result in small changes in the PIM vector.
3. Completeness: The transformation captures all information about polarity transitions that is relevant to protein function.
4. Invariance: The transformation is invariant under permutations of the sequence that preserve the transition frequencies.

Definition 3.1.1 (Amino Acid Polarity). *Let $\mathcal{A} = \{A_1, \dots, A_{20}\}$ be the set of the 20 standard amino acids. We define the polarity function $\phi : \mathcal{A} \rightarrow \{P^+, P^-, N, NP\}$ as:*

$$\phi(A) = \begin{cases} P^+ & \text{if } A \in \{H, K, R\} \\ P^- & \text{if } A \in \{D, E\} \\ N & \text{if } A \in \{C, G, N, Q, S, T, Y\} \\ NP & \text{if } A \in \{A, F, I, L, M, P, V, W\} \end{cases} \quad (3.1)$$

Definition 3.1.2 (PIM Vector). For a protein sequence $S = s_1 s_2 \dots s_L$ of length L , we define the PIM vector $v \in \mathbb{R}^{16}$ as:

$$v_i = \frac{1}{L-1} \sum_{k=1}^{L-1} \mathbf{1}\{\phi(s_k) = a, \phi(s_{k+1}) = b\} \quad (3.2)$$

where i is the index corresponding to the transition $a \rightarrow b$ in the order of the 16 possible transitions.

Example 3.1.1. Let's compute the PIM vector for a simple protein sequence: **AKDR**. First, we classify each amino acid:

- *A (Alanine) $\rightarrow$ NP (Nonpolar)*
- *K (Lysine) $\rightarrow$ P^+ (Positively charged)*
- *D (Aspartic acid) $\rightarrow$ P^- (Negatively charged)*
- *R (Arginine) $\rightarrow$ P^+ (Positively charged)*

The polarity string is: **NP, P^+ , P^- , P^+**

Now we count the transitions:

- *NP $\rightarrow$ P^+ : 1 transition*
- *$P^+ \rightarrow P^-$: 1 transition*
- *$P^- \rightarrow P^+$: 1 transition*

Normalizing by the total number of transitions ($L-1=3$), we get the PIM vector $v \in \mathbb{R}^{16}$:

The indices for the transitions are:

- *NP $\rightarrow$ P^+ : index 12 (0-based counting: NP=3, $P^+=0$, so $3 \times 4 + 0 = 12$)*
- *$P^+ \rightarrow P^-$: index 1 ($0 \times 4 + 1 = 1$)*
- *$P^- \rightarrow P^+$: index 4 ($1 \times 4 + 0 = 4$)*

Thus:

$$v_{12} = 1/3, \quad v_1 = 1/3, \quad v_4 = 1/3 \quad (3.3)$$

and all other components are 0.

Solution: The PIM vector is:

$$v = [0 \quad 1/3 \quad 0 \quad 0 \quad 1/3 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1/3 \quad 0 \quad 0 \quad 0]^T \quad (3.4)$$

□

3.2 The 16 Polarity Interactions

The 16 possible polarity transitions are:

Table 3.1: The 16 polarity transitions with indices

Index	Transition	Index	Transition
0	$P^+ \rightarrow P^+$	8	$N \rightarrow P^+$
1	$P^+ \rightarrow P^-$	9	$N \rightarrow P^-$
2	$P^+ \rightarrow N$	10	$N \rightarrow N$
3	$P^+ \rightarrow NP$	11	$N \rightarrow NP$
4	$P^- \rightarrow P^+$	12	$NP \rightarrow P^+$
5	$P^- \rightarrow P^-$	13	$NP \rightarrow P^-$
6	$P^- \rightarrow N$	14	$NP \rightarrow N$
7	$P^- \rightarrow NP$	15	$NP \rightarrow NP$

3.3 Biological Weights

The system uses biological weights to emphasize physiologically relevant interactions:

Table 3.2: Biological weights for polarity interactions

Interaction	Weight
P^+, P^-	2.0
P^-, P^+	2.0
N, N	1.5
N, P^+	1.3
P^+, N	1.3
N, P^-	1.3
P^-, N	1.3
NP, NP	1.0
NP, N	0.9
N, NP	0.9
NP, P^+	0.7
P^+, NP	0.7
NP, P^-	0.7
P^-, NP	0.7
P^+, P^+	0.4
P^-, P^-	0.4

3.4 Metric Signature

The metric signature defines the geometry of the PIM space:

$$\text{METRIC_SIGNATURE} = \begin{cases} [-1, +1, +1, 0, +1, -1, +1, 0, +1, +1, +1, 0, 0, 0, 0, +1] & \text{if USE_BIOLOGIC} \\ [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1] & \text{if not} \end{cases} \quad (3.5)$$

3.5 Functions for PIM Computation

This section describes the key functions used for computing PIM profiles and related features.

compute_pim_profile

This function converts a protein sequence into a PIM vector:

$$\text{pim_profile}(S) = \frac{1}{L-1} \sum_{k=1}^{L-1} \mathbf{1}\{\phi(s_k) \rightarrow \phi(s_{k+1})\} \quad (3.6)$$

The function also applies biological weights if `USE_WEIGHTS` is set to `True`. The weighted vector is normalized so that all components sum to one.

compute_trimer_profile

This function computes the trimer profile for a sequence:

$$\text{trimer_profile}(S) = \frac{1}{L-2} \sum_{k=1}^{L-2} \mathbf{1}\{\phi(s_k) \rightarrow \phi(s_{k+1}) \rightarrow \phi(s_{k+2})\} \quad (3.7)$$

GRASSMANN ALGEBRA: FUNDAMENTAL OPERATIONS

This chapter presents the fundamental operations of Grassmann algebra as implemented in SGPMAN (10.0 versión), including the wedge product, specular reflection, Clifford signature, and other key operations.

4.1 Introduction to Grassmann Algebra

Grassmann algebra, also known as exterior algebra, provides a natural mathematical framework for working with vector subspaces and their geometric relationships. The fundamental operation in Grassmann algebra is the wedge product, which generalizes the cross product to arbitrary dimensions. This is essential for protein analysis because it allows us to measure the geometric relationship between two PIM vectors in a way that captures both their similarity and their orthogonality.

The key insight of Grassmann algebra is that while vectors represent points or directions, the wedge product of two vectors represents a plane (or more generally, a subspace). The magnitude of the wedge product gives the area of the parallelogram spanned by the vectors, and its direction gives the orientation of that parallelogram. This geometric interpretation is particularly powerful for protein analysis because it allows us to think of PIM vectors as directions in a 16-dimensional space, and the wedge product as a measure of how aligned those directions are.

4.2 Wedge Product and Corrected Similarity

Mathematical Definition and Interpretation

The wedge product of two PIM vectors captures the geometric relationship between them. The full wedge product in 16 dimensions would have 120 components, but we use a truncated version that focuses on the six biologically most relevant pairs of transitions. This truncation is justified because the six selected pairs capture the essential physicochemical information needed for functional comparison.

Definition 4.2.1 (Truncated Wedge Product). *Let $v, w \in \mathbb{R}^{16}$ be two PIM vectors. We define the truncated wedge product over the set of biologically relevant pairs $\mathcal{K}$:*

$$\mathcal{K} = \{(0, 5), (1, 4), (2, 6), (3, 7), (10, 11), (14, 15)\} \quad (4.1)$$

The truncated wedge product is:

$$\omega_{\mathcal{K}}(v, w)_k = v_{i_k} w_{j_k} - v_{j_k} w_{i_k} \quad \text{for } (i_k, j_k) \in \mathcal{K} \quad (4.2)$$

Definition 4.2.2 (Corrected Wedge Similarity). *The corrected wedge similarity between two PIM vectors $v, w \in \mathbb{R}^{16}$ is defined as:*

$$\text{WedgeSim}(v, w) = 1 - \frac{\|\omega_{\mathcal{K}}(v, w)\|_2}{\|v\|_2 \|w\|_2} \quad (4.3)$$

This similarity satisfies $0 \leq \text{WedgeSim}(v, w) \leq 1$, where $\text{WedgeSim}(v, w) = 1$ indicates identical vectors and $\text{WedgeSim}(v, w) = 0$ indicates orthogonal vectors.

Remark 4.2.1. *The corrected wedge similarity is a more stringent measure than the uncorrected wedge similarity used in previous versions. A value of 0.12 indicates near-orthogonality of the polarity profiles, representing a substantial functional difference between the compared proteins. This improved discriminative power is the key advance of version 9.0.*

4.3 Specular Reflection

The Concept of Charge Inversion

Specular reflection is a novel concept introduced in SGPMAIN 8.0 that captures the idea of charge-inverted functional equivalence. The insight is that two proteins might have similar functions even if their charge patterns are inverted—a phenomenon that occurs in some cases of molecular mimicry and immune evasion. The mathematical operation of specular reflection inverts the charge components of the PIM vector, allowing us to test whether two proteins are related by charge inversion.

The mathematical properties of specular reflection ensure that it is a well-defined geometric transformation:

1. The reflection is an involution (applying it twice returns the original vector).
2. The reflection preserves the Euclidean norm (it is an isometry).
3. The reflection is self-adjoint (it preserves the inner product structure).

Definition 4.3.1 (Reflection Normal Vector). *The normal vector $n \in \mathbb{R}^{16}$ for specular reflection is defined as:*

$$n_i = \begin{cases} +1 & \text{if } i \in \{0, 1, 4, 5\} \text{ (charge components)} \\ 0 & \text{otherwise} \end{cases} \quad (4.4)$$

normalized as $n = n/\|n\|_2$.

Definition 4.3.2 (Specular Reflection). *The specular reflection of a PIM vector $v \in \mathbb{R}^{16}$ is defined as:*

$$R_c(v) = v - 2(v \cdot n)n \quad (4.5)$$

This transformation inverts the sign of the charge components, simulating a charge inversion (P^+ to P^- and P^- to P^+).

4.4 Clifford Signature

Comprehensive Characterization

The Clifford signature provides a comprehensive characterization of a protein's polarity profile by combining multiple geometric, information-theoretic, and statistical measures into a single vector. Each component of the signature captures a different aspect of the polarity profile, and together they form a fingerprint that can be used for classification, comparison, and functional prediction.

The components of the Clifford signature are chosen to be:

1. Informative: Each component provides unique information about the polarity profile.
2. Interpretable: Each component has a clear biological meaning.
3. Stable: Each component is robust to small changes in the sequence.
4. Discriminative: Each component helps to distinguish between different functional classes of proteins.

Definition 4.4.1 (Clifford Signature). *The Clifford signature of a PIM vector $v \in \mathbb{R}^{16}$ is the 12-component vector:*

$$\Sigma(v) = (\|v\|, \text{AutoRef}(v), \text{HydroProj}(v), \text{ChargeProj}(v), \text{AutoRot}(v), \\ H(v), D_f(v), G(v), C(v), M(v), I(v), \Lambda(v)) \quad (4.6)$$

where:

$$\text{AutoRef}(v) = \text{WedgeSim}(v, R_c(v)) \quad (4.7)$$

$$\text{HydroProj}(v) = |v_{10} + v_{15}| \quad (4.8)$$

$$\text{ChargeProj}(v) = |v_0 + v_5| \quad (4.9)$$

$$\text{AutoRot}(v) = \text{WedgeSim}(v, \text{roll}(v, 4)) \quad (4.10)$$

THE 18 MATHEMATICAL METRICS

This chapter presents the complete mathematical description of all 18 metrics implemented in SGPMAIN (10.0 versión). These metrics are derived from the PIM vectors and provide a comprehensive characterization of protein polarity profiles. Each metric captures a different aspect of the polarity distribution, from information-theoretic measures to geometric and topological properties.

5.1 Shannon Entropy

The Shannon entropy measures the uncertainty or randomness in the polarity distribution. A higher entropy indicates a more uniform distribution of polarity transitions, while a lower entropy indicates that a few transitions dominate. This metric is fundamental for characterizing the diversity of polarity interactions in a protein.

Definition 5.1.1 (Shannon Entropy). *For a PIM vector $v \in \mathbb{R}^{16}$, the Shannon entropy is:*

$$H(v) = - \sum_i \frac{|v_i|}{\|v\|_1} \log_2 \left(\frac{|v_i|}{\|v\|_1} + \epsilon \right) \quad (5.1)$$

where $\|v\|_1 = \sum_i |v_i|$ is the L1 norm, and $\epsilon = 10^{-10}$ is a small constant to avoid division by zero.

The entropy ranges from 0 to $\log_2(16) = 4$ bits.

5.2 Jensen-Shannon Divergence

The Jensen-Shannon divergence measures the similarity between two probability distributions. It is a symmetric and smoothed version of the Kullback-Leibler divergence. Unlike the KL divergence, the JS divergence is symmetric and always finite, making it particularly suitable for comparing protein polarity profiles.

Definition 5.2.1 (Jensen-Shannon Divergence). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Jensen-Shannon divergence is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon}, \quad q = \frac{|w|}{\|w\|_1 + \epsilon} \quad (5.2)$$

$$m = \frac{p + q}{2} \quad (5.3)$$

$$JS(v, w) = \frac{1}{2} \sum_i p_i \log_2 \left(\frac{p_i}{m_i} \right) + \frac{1}{2} \sum_i q_i \log_2 \left(\frac{q_i}{m_i} \right) \quad (5.4)$$

The JS divergence ranges from 0 (identical distributions) to 1 (completely different distributions).

5.3 Gini Coefficient

The Gini coefficient measures the inequality in the polarity distribution. A lower Gini coefficient indicates a more uniform distribution, while a higher coefficient indicates that the distribution is concentrated in a few transitions. This metric is particularly useful for identifying proteins with highly specialized polarity patterns.

Definition 5.3.1 (Gini Coefficient). *For a PIM vector $v \in \mathbb{R}^{16}$, the Gini coefficient is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon} \quad (5.5)$$

$$G(v) = 1 - \frac{2}{n} \sum_{i=1}^n \frac{\sum_{j \leq i} p_{(j)}}{\sum_j p_{(j)}} \quad (5.6)$$

where $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(n)}$ are the sorted probabilities, and $n = 16$.

The Gini coefficient ranges from 0 (perfect equality) to 1 (perfect inequality).

5.4 Structural Complexity Index

The structural complexity index combines multiple measures to capture the overall complexity of the polarity profile. This composite metric integrates entropy, fractal dimension, and richness to provide a single value that reflects the structural complexity of the protein's polarity organization.

Definition 5.4.1 (Structural Complexity Index). *For a PIM vector $v \in \mathbb{R}^{16}$, the structural complexity index is:*

$$C(v) = 0.4H_{norm}(v) + 0.35D_f(v) + 0.25R(v) \quad (5.7)$$

where:

- $H_{norm}(v) = H(v)/\log_2(16)$ is the normalized Shannon entropy
- $D_f(v)$ is the fractal dimension (see Section 7.13)
- $R(v) = nnz(v)/16$ is the richness measure (number of non-zero components)

The complexity index ranges from 0 to 1.

5.5 Functional Modularity Index

The functional modularity index measures the degree of clustering in the polarity transitions. This metric identifies whether significant polarity transitions are organized into distinct clusters, which may correspond to functional domains or regions of the protein.

Definition 5.5.1 (Functional Modularity Index). *For a PIM vector $v \in \mathbb{R}^{16}$ and a threshold $\theta = 0.05$, the functional modularity index is:*

1. Identify significant transitions: $S = \{i \mid |v_i| > \theta\}$
2. Cluster significant transitions by proximity: clusters are formed when indices differ by at most 2
3. Compute cluster energies: $E_k = \sum_{i \in cluster_k} |v_i|$
4. Normalize: $\hat{E}_k = E_k / \sum_j E_j$
5. Compute entropy: $H_{cluster} = -\sum_k \hat{E}_k \log_2(\hat{E}_k)$

$$M(v) = 1 - \frac{H_{cluster}}{\log_2(K)} \quad (5.8)$$

where K is the number of clusters. The modularity index ranges from 0 (no modularity) to 1 (perfect modularity).

5.6 Moran's I

Moran's I measures the spatial autocorrelation in the polarity profile. Positive values indicate clustering of similar values, negative values indicate dispersion, and zero indicates random distribution. This metric captures the spatial organization of polarity transitions along the sequence.

Definition 5.6.1 (Moran's I). *For a PIM vector $v \in \mathbb{R}^{16}$, Moran's I is:*

$$z_i = v_i - \bar{v} \quad (5.9)$$

$$w_{ij} = \exp\left(-\frac{|i-j|}{2}\right) \quad (5.10)$$

$$I(v) = \frac{n}{\sum_{i,j} w_{ij}} \frac{\sum_{i,j} w_{ij} z_i z_j}{\sum_i z_i^2} \quad (5.11)$$

where $n = 16$ is the number of components, and $\bar{v}$ is the mean of v .

5.7 Hellinger Distance

The Hellinger distance measures the similarity between two probability distributions. It is a metric that satisfies the triangle inequality and is bounded between 0 and 1, making it particularly suitable for comparing protein polarity profiles.

Definition 5.7.1 (Hellinger Distance). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Hellinger distance is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon}, \quad q = \frac{|w|}{\|w\|_1 + \epsilon} \quad (5.12)$$

$$H_d(v, w) = \frac{1}{\sqrt{2}} \|\sqrt{p} - \sqrt{q}\|_2 \quad (5.13)$$

The Hellinger distance ranges from 0 (identical distributions) to 1 (completely different distributions).

5.8 Spearman Correlation

The Spearman correlation measures the rank correlation between two PIM vectors. Unlike the Pearson correlation, the Spearman correlation is non-parametric and captures monotonic relationships, making it robust to outliers and non-linear relationships.

Definition 5.8.1 (Spearman Correlation). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Spearman correlation is:*

$$r_i = \text{rank}(v_i), \quad s_i = \text{rank}(w_i) \quad (5.14)$$

$$\rho(v, w) = \frac{\sum_i (r_i - \bar{r})(s_i - \bar{s})}{\sqrt{\sum_i (r_i - \bar{r})^2 \sum_i (s_i - \bar{s})^2}} \quad (5.15)$$

The Spearman correlation ranges from -1 (perfect negative correlation) to 1 (perfect positive correlation).

5.9 Grassmann Projection Distance

The Grassmann projection distance measures the distance between two subspaces in the Grassmann manifold. This metric captures the geometric relationship between the subspaces spanned by two PIM vectors, providing a measure of their structural similarity.

Definition 5.9.1 (Grassmann Projection Distance). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Grassmann projection distance is:*

$$P_v = \frac{vv^T}{\|v\|^2}, \quad P_w = \frac{ww^T}{\|w\|^2} \quad (5.16)$$

$$d_{Gr}(v, w) = \frac{1}{\sqrt{2}} \|P_v - P_w\|_F \quad (5.17)$$

where $\|\cdot\|_F$ is the Frobenius norm. The distance ranges from 0 (identical subspaces) to 1 (orthogonal subspaces).

5.10 Fubini-Study Metric

The Fubini-Study metric measures the distance in projective space. This metric captures the angle between two vectors in the projective space, providing a measure of their directional similarity.

Definition 5.10.1 (Fubini-Study Metric). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Fubini-Study metric is:*

$$d_{FS}(v, w) = \arccos \left(\frac{|v \cdot w|}{\|v\|_2 \|w\|_2} \right) \quad (5.18)$$

The Fubini-Study metric ranges from 0 (identical directions) to $\pi/2$ (orthogonal directions).

5.11 Ricci Curvature

The Ricci curvature measures the geometric curvature of the space between two vectors. This metric captures the local geometry of the PIM space and can indicate regions of high or low curvature that may correspond to functional significance.

Definition 5.11.1 (Ricci Curvature). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Ricci curvature is:*

$$\theta = \arccos \left(\frac{|v \cdot w|}{\|v\|_2 \|w\|_2} \right) \quad (5.19)$$

$$\kappa(v, w) = \frac{1}{\tan^2(\theta) + \epsilon} \quad (5.20)$$

where $\epsilon = 10^{-10}$ is a small constant to avoid division by zero. Higher curvature indicates more curved geometry.

5.12 Wasserstein Distance

The Wasserstein-1 distance measures the optimal transport cost between two distributions. This metric captures the minimal cost of transforming one polarity distribution into another, providing a meaningful measure of distributional similarity.

Definition 5.12.1 (Wasserstein-1 Distance). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Wasserstein-1 distance is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon}, \quad q = \frac{|w|}{\|w\|_1 + \epsilon} \quad (5.21)$$

$$CDF_v(i) = \sum_{j=1}^i p_j, \quad CDF_w(i) = \sum_{j=1}^i q_j \quad (5.22)$$

$$W_1(v, w) = \sum_{i=1}^n |CDF_v(i) - CDF_w(i)| \quad (5.23)$$

where $n = 16$. The Wasserstein distance ranges from 0 (identical distributions) to 16 (completely different distributions).

5.13 Fractal Dimension

The fractal dimension measures the self-similarity of the polarity profile. This metric captures the scale-invariant properties of the polarity distribution, providing insight into its structural complexity.

Definition 5.13.1 (Fractal Dimension). *For a PIM vector $v \in \mathbb{R}^{16}$, the fractal dimension is:*

1. Compute the absolute values: $a_i = |v_i|$
2. Compute the cumulative sum: $c_i = \sum_{j=1}^i a_j$
3. Normalize: $\hat{c}_i = (c_i - \min c) / (\max c - \min c)$
4. For each scale $s = 1, \dots, S$, compute box count: $N_s = \#\{\text{unique boxes}\}$

$$D_f(v) = -\frac{\log(N_s)}{\log(s)} \quad (5.24)$$

The fractal dimension ranges from 0 to 1.

5.14 Radon Transform

The Radon transform projects the polarity profile onto different angles. This transformation captures the directional properties of the polarity distribution, providing a representation that is invariant to certain types of transformations.

Definition 5.14.1 (Radon Transform). *For a PIM vector $v \in \mathbb{R}^{16}$, the Radon transform is:*

$$\mathcal{R}(v)(\theta_k) = \sum_{i=1}^n v_i^2 \cos^2(i\theta_k) \quad (5.25)$$

where $\theta_k = k\pi/n$ for $k = 0, \dots, n-1$, and $n = 16$.

The Radon similarity between two vectors is:

$$\text{RadonSim}(v, w) = \frac{\mathcal{R}(v) \cdot \mathcal{R}(w)}{\|\mathcal{R}(v)\|_2 \|\mathcal{R}(w)\|_2} \quad (5.26)$$

5.15 Karhunen-Loève Decomposition

The Karhunen-Loève decomposition (also known as Principal Component Analysis) identifies the most informative components of the polarity profile. This decomposition provides a low-dimensional representation that preserves the essential information about the polarity distribution.

Definition 5.15.1 (Karhunen-Loève Decomposition). *Let $\{v_1, \dots, v_n\}$ be a set of PIM vectors. The covariance matrix is:*

$$C = \frac{1}{n-1} \sum_{i=1}^n (v_i - \bar{v})(v_i - \bar{v})^T \quad (5.27)$$

where $\bar{v}$ is the mean vector.

The Karhunen-Loève decomposition solves the eigenvalue problem:

$$C\phi_k = \lambda_k\phi_k, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{16} \geq 0 \quad (5.28)$$

The explained variance for component k is:

$$\text{EV}_k = \frac{\lambda_k}{\sum_{j=1}^{16} \lambda_j} \quad (5.29)$$

The cumulative explained variance for the first K components is:

$$\text{EV}_{cum}(K) = \sum_{k=1}^K \text{EV}_k \quad (5.30)$$

5.16 Polarity Laplacian

The polarity Laplacian measures the connectivity structure of the polarity profile. This metric captures the graph-theoretic properties of the polarity transitions, providing insight into the network structure of polarity interactions.

Definition 5.16.1 (Polarity Laplacian). *For a PIM vector $v \in \mathbb{R}^{16}$, the polarity Laplacian is:*

$$L_{ij} = \begin{cases} \sum_{k \neq i} v_i v_k & \text{if } i = j \\ -v_i v_j & \text{if } i \neq j \end{cases} \quad (5.31)$$

5.17 Commutator Norm

The commutator norm measures the non-commutativity of two PIM vectors. This metric captures the degree to which the wedge product fails to commute, providing a measure of the geometric incompatibility between two polarity profiles.

Definition 5.17.1 (Commutator Norm). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the commutator norm is:*

$$[v, w] = v \wedge w - w \wedge v = 2(v \wedge w) \quad (5.32)$$

$$\|[v, w]\| = \frac{\|v \wedge w\|_2}{\|v\|_2 \|w\|_2} \quad (5.33)$$

The commutator norm ranges from 0 (commuting vectors) to 1 (completely non-commuting vectors).

5.18 Anticommutator Similarity

The anticommutator similarity measures the commutativity of two PIM vectors. This metric captures the degree to which the dot product captures the relationship between two polarity profiles.

Definition 5.18.1 (Anticommutator Similarity). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the anticommutator similarity is:*

$$\{v, w\} = v \cdot w + w \cdot v = 2(v \cdot w) \quad (5.34)$$

$$\text{anticommutator_sim}(v, w) = \frac{|v \cdot w|}{\|v\|_2 \|w\|_2} \quad (5.35)$$

The anticommutator similarity ranges from 0 (orthogonal vectors) to 1 (parallel vectors).

HOW METRICS INTERACT WITH **CONFIG.JSON**

This chapter explains how the 18 mathematical metrics interact with the **config.json** file in **SGPMAIN (10.0 versión)**. The **config.json** file provides target-specific parameters that guide the application of the metrics, making the system adaptable to different pathogens.

6.1 The Role of Metric Weights

The metric weights in **config.json** determine the relative importance of each metric in the peptide design process. The weights are used in the composite score calculation:

$$\text{Composite} = \sum_{m \in \mathcal{M}} w_m \cdot \Delta_m \quad (6.1)$$

where $\mathcal{M}$ is the set of metrics, w_m are the weights from **config.json**, and Δ_m are the differences between the target and host.

For West Nile virus, the weights might be:

Table 6.1: Metric weights for West Nile virus (from **config.json**)

Metric	Weight	Rationale
PIM	0.28	High importance for polarity profiling
Entropy	0.12	Moderate importance
Grassmann	0.14	High importance for geometric analysis
Hodge	0.08	Moderate importance
Curvature	0.06	Lower importance
Gini	0.04	Lower importance
Fubini-Study	0.04	Lower importance
Jensen-Shannon	0.06	Moderate importance
Spearman	0.04	Lower importance
Hellinger	0.04	Lower importance
Wasserstein	0.04	Lower importance
Fractal	0.03	Lower importance
Radon	0.03	Lower importance

6.2 The Role of Target-Specific Ranges

The target-specific ranges in **config.json** define the acceptable values for each metric when validating a designed peptide. These ranges ensure that the peptide is not just similar to the target in the composite score, but also meets specific criteria for each metric.

For West Nile virus, the ranges might be:

Table 6.2: Target-specific ranges for West Nile virus (from **config.json**)

Metric	Min	Max
PIM Similarity	0.72	0.90
Antiviral Activity	0.68	1.00
Selectivity Index	40	100
Entropy	2.5	3.6
Grassmann Distance	0.30	0.55
Hodge Complementarity	0.68	0.92
Ricci Curvature	0.25	0.65
Jensen-Shannon	0.15	0.45
Hellinger	0.15	0.40
Wasserstein	0.20	0.55
Fractal Dimension	1.2	2.1
Drug Likeness	0.78	1.00

6.3 How the Validation Works

When a peptide is designed, the system validates it against the target-specific ranges from **config.json**:

1. The peptide's PIM vector is computed.
2. For each metric, the system calculates the value.
3. The system checks if the value falls within the range defined in **config.json**.
4. If all metrics are within range, the peptide passes validation.
5. If any metric is outside its range, the peptide fails validation and is flagged for optimization.

This validation ensures that the designed peptide is not just a good match in the composite score, but also meets specific criteria for each individual metric.

6.4 Why This Matters for Peptide Design

The interaction between the metrics and `config.json` is what makes SGPMAIN (10.0 versión) so powerful for peptide design:

Adaptability: Different pathogens have different characteristics. The ability to adjust metric weights and ranges for each target means that the system can be optimized for the specific features of each virus.

Specificity: The target-specific ranges ensure that the designed peptide is not just generically similar to the target, but meets specific criteria that are important for that particular pathogen.

Validation: The validation process provides a clear pass/fail criterion for each metric, making it easy to identify which aspects of the peptide need optimization.

Transparency: The `config.json` file makes all parameters explicit and adjustable, providing full transparency into how the system is making decisions.

Part III

Case Study: West Nile Virus

APPLICATION TO WEST NILE VIRUS

This chapter demonstrates the application of SGPMAIN (10.0 versión) to West Nile virus (WNV) envelope proteins. The case study shows how the system processes sequences, computes metrics, and designs therapeutic peptides.

7.1 Introduction to West Nile Virus

West Nile virus (WNV) is a mosquito-borne flavivirus that causes febrile illness and neurological disease in humans. The virus encodes two key envelope proteins that are essential for viral entry and pathogenesis: the small envelope protein M (NILE1) and the envelope glycoprotein E (NILE2). These proteins are the focus of this case study.

7.2 NILE1: Small Envelope Protein M

The membrane protein M (NILE1) is a small, hydrophobic protein that is cleaved from the precursor prM during viral maturation. It plays a crucial role in membrane fusion and virus entry.

Sequence

```
>P06935_005 West Nile virus; 11082; 216-290; Small envelope protein M
SLTVQTHGESTLANKKGAWLDSTKATRYLVKTESWILRNPGYALVAAVIGWMLGSNTMQR
VVFALLLLLVPAYS
```

Key Properties

Table 7.1: Key properties of NILE1

Property	Value
Length	75 amino acids
Hydrophobicity	High (contains transmembrane domain)
Function	Membrane fusion, virus entry
Structure	Predominantly α -helical

7.3 NILE2: Envelope Glycoprotein E

The envelope glycoprotein E (NILE2) is the major surface protein of WNV. It is responsible for receptor binding and membrane fusion.

Sequence

```
>P06935_006 West Nile virus; 11082; 291-787; Envelope protein E
FNCLGMSNRDFLEGVSGATWVDLVLEGDSCVTIMSKDKPTIDVKMMNMEAANLADVRSYC
YLASVSDLSTRAACPTMGEAHNEKRADPAFVCKQGVDVDRGWGNGCGLFGKGSIDTCAKFA
CTTKATGWIIQKENIKYEVAIFVHGPTTVESHGKIGATQAGRFSITPSAPSYTLKLGEYG
EVTVDCEPRSGIDTSAYYVMSVGEKSFLVHREWFMDLNLPSWSSAGSTTWRNRETLMEFEE
PHATKQSVVALGSQEGALHQALAGAIPVEFSSNTVKLTSGHLKCRVKMEKLQLKGTTYGV
CSKAFKFARTPADTGHGTVVLELQYTGTDGPCKVPISSVASLNDLTPVGRLVTVPFVSV
ATANSKVLIELEPPFGDSYIVVGRGEQQINHHWHKSGSSIGKAFTTTLRGAQRLAALGDT
AWDFGSVGGVFTSVGKAIHQVFGGAFRSLFGGMSWITQGLLGALLLWMGINARDRSIAMT
FLAVGGVLLFLSVNVHA
```

Key Properties

Table 7.2: Key properties of NILE2

Property	Value
Length	497 amino acids
Structure	Three domains (DI, DII, DIII)
Function	Receptor binding, membrane fusion, immune evasion
Key residues	Fusion loop (98-109), receptor binding domain

7.4 PIM Profiles for WNV Proteins

The PIM profiles for the two WNV proteins reveal their distinct polarity organizations:

Table 7.3: Key PIM components for NILE1 and NILE2

Transition	NILE1 (M)	NILE2 (E)
NP→NP	0.467	0.298
NP→N	0.040	0.067
N→NP	0.027	0.066
P ⁺ →P ⁻	0.000	0.011
P ⁻ →P ⁺	0.000	0.006
NP→P ⁺	0.013	0.031

7.5 Similarity Analysis: NILE1 vs NILE2

The wedge similarity between the two WNV proteins is:

$$\text{WedgeSim}(v_{\text{NILE1}}, v_{\text{NILE2}}) = 0.1264 \quad (7.1)$$

This low similarity value indicates that the two proteins have substantially different polarity profiles, consistent with their different functions: NILE1 is a small membrane protein, while NILE2 is a large surface glycoprotein.

7.6 WNV Chemical Profile Summary

Table 7.4: Selected chemical properties for NILE1 and NILE2

Property	NILE1 (M)	NILE2 (E)
Net Charge	0.267	0.052
Charge Density	0.093	0.085
Charge Balance	0.000	0.808
pI	6.867	5.974
GRAVY	0.369	0.253
N-Glycosylation Sites	5	12
O-Glycosylation Sites	22	64
Phosphorylation Sites	10	32
Solubility (mg/mL)	7.29	12.81
ΔG (kcal/mol)	-4.958	-6.760
Tm (°C)	68.85	41.04
Stability Score	0.780	0.552
Best Lipid Binding	Cholesterol	Cholesterol

7.7 Hotspot Analysis

Three drug-binding hot spots were identified in NILE2:

Table 7.5: Hotspots identified in NILE2

Hotspot Name	Score	Key Residues
Charge-rich (CR2-like)	0.104	H54, E64, D66, K68, H71
Hydrophobic (fusion loop-like)	0.339	D512, E515, H516, I518, W519, A520
Mixed polar/hydrophobic (binding loop-like)	0.268	F134, F136, Y139, D142, K144

7.8 PIDP Analysis for WNV Proteins

The PIDP analysis provides critical insight into the structural flexibility of the WNV proteins:

Table 7.6: PIDP results for NILE1 and NILE2

Metric	NILE1 (M)	NILE2 (E)
metapredict disorder (0.5)	10.2%	8.15%
metapredict mean score	0.1912	0.1655
AIUPred disorder (0.5)	11.8%	9.69%
AIUPred mean score	0.2045	0.1919
AIUPred redox sensitivity	0.015	0.029

Interpretation: The low disorder percentages (8-12%) confirm that both WNV envelope proteins are predominantly ordered, consistent with their structural roles. NILE2 shows slightly lower disorder (8.15%) than NILE1 (10.2%), which is expected given that NILE2 has a well-defined three-domain structure. The redox sensitivity values are low, indicating that the disorder profiles are relatively stable under different redox conditions.

Why This Matters: The low disorder of these proteins suggests that they are suitable targets for structure-based drug design and that they maintain stable conformations during viral entry and fusion. This information is crucial for understanding the mechanism of viral entry and for designing effective therapeutic interventions.

7.9 Peptide Design for WNV

The peptide design engine generated candidate peptides targeting WNV envelope protein E:

Table 7.7: Top designed peptides targeting WNV E protein

Rank	Sequence	Drug Score	Activity	Recommendation
1	FDLGNNKAAAV	0.823	0.853	PROMETEDOR
2	FDLGNNKAATV	0.801	0.842	PROMETEDOR
3	FDLGNNKAAVV	0.789	0.831	MODERADO
4	FDLGNNKAAAV	0.776	0.818	MODERADO
5	FDLGNNKAAAV	0.761	0.805	MODERADO

Best Candidate Rationale

DESIGNED PEPTIDE: FDLGNNKAAAV
=====

PREDICTIVE PROFILE:

- Antiviral activity: 0.853
- Selectivity index: 82.4
- Stability: 14.2 hours
- Toxicity: CC50 = 48.3 μ M
- Hemolysis: 3.8%

DRUG-LIKENESS: 0.823

- Lipinski_modified: Passed
- Veber_rules: Passed
- Toxicity_filters: Passed
- Stability_filters: Passed
- Synthesis_feasibility: Passed

STRUCTURAL CHARACTERISTICS:

- Length: 11 aa
- Composition: F(1), D(1), L(1), G(1), N(2), K(1), A(3), V(1)

CONFIGURATION USED:

- Source: config.json (West Nile virus specific)
- Metric weights: PIM(0.28), Grassmann(0.14), Entropy(0.12), etc.
- Target ranges: Validated against WNV-specific thresholds

Note: The peptide design process used parameters from config.json, including target-specific metric weights and ranges. This ensures that the designed peptide is optimized specifically for West Nile virus.

INSTALLATION AND SYSTEM REQUIREMENTS

This appendix provides concise information on system requirements, installation steps, and how to run SGPMAN (10.0 versión).

A.1 System Requirements

Table A.1: Minimum system requirements

Component	Requirement
Operating System	Linux, macOS, or Windows (with WSL)
Python	Version 3.8 or higher
RAM	8 GB (32 GB recommended for ESM2)
Storage	20 GB free space
Processor	64-bit, 4 or more cores
GPU (optional)	NVIDIA GPU with 4+ GB VRAM

A.2 Required Packages

```
numpy pandas scipy scikit-learn matplotlib seaborn psutil
torch transformers peft tokenizers accelerate
```

A.3 Installation Steps

1. Clone the repository:

```
git clone https://github.com/polancodf2024/nile-sgp-analysis.git
cd nile-sgp-analysis
```

2. Install core dependencies:

```
pip install numpy pandas scipy scikit-learn matplotlib seaborn psutil
```

3. Install deep learning dependencies:

```
pip install torch transformers peft tokenizers accelerate
```

4. (Optional) Install PIDP dependencies:

```
pip install metapredict aiupred
```

A.4 Running SGPMMAIN (10.0 versión)

1. Place your FASTA files in the working directory:

```
nile1.unico.dat0 nile2.unico.dat0 ...
```

2. **(Optional) Place config.json in the working directory.** If present and valid, the program will use its parameters. If missing, it will fall back to default values.
3. Run the program:

```
python3 SGPMMAIN10.0v.py
```

4. Results are saved in:

```
results_v91_YYYYMMDD_HHMMSS/
```

A.5 Quick Reference

```
# Run full analysis
python3 SGPMMAIN10.0v.py
```

```
# View results
ls results_v91_*/
```

```
# View summary
cat results_v91_*/FINAL_SUMMARY.txt
```

```
# View best candidate
cat results_v91_*/best_candidate.json
```

BIBLIOGRAPHY

- [1] Polanco, C., Uversky, V.N., Huberman, A., et al. (2023). Bioinformatics-based Analysis of the Variability of MPOX Virus Proteins. *Current Bioinformatics*, 18(15), 170-191.
- [2] Hestenes, D., & Sobczyk, G. (1984). *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel Publishing Company.
- [3] Hestenes, D. (2017). The geometric algebra of Grassmann and Clifford. *Advances in Applied Clifford Algebras*, 27(1), 101-117.
- [4] Dorst, L., Fontijne, D., & Mann, S. (2007). *Geometric Algebra for Computer Science: An Object-Oriented Approach to Geometry*. Morgan Kaufmann.
- [5] Edelman, A., Arias, T. A., & Smith, S. T. (1998). The geometry of algorithms with orthogonality constraints. *SIAM Journal on Matrix Analysis and Applications*, 20(2), 303-353.
- [6] Fubini, G. (1904). Sulle metriche definite da una forma Hermitiana. *Atti del Reale Istituto Veneto di Scienze, Lettere ed Arti*, 63(2), 501-513.
- [7] Study, E. (1905). Kürzeste Wege im komplexen Gebiet. *Mathematische Annalen*, 60(3), 321-378.
- [8] Radon, J. (1917). Über die Bestimmung von Funktionen durch ihre Integralwerte längs gewisser Mannigfaltigkeiten. *Berichte über die Verhandlungen der Königlich-Sächsischen Gesellschaft der Wissenschaften zu Leipzig, Mathematisch-Physische Klasse*, 69, 262-277.
- [9] Mandelbrot, B. B. (1983). *The Fractal Geometry of Nature*. W. H. Freeman.
- [10] Householder, A. S. (1958). Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4), 339-342.
- [11] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423.
- [12] Jensen, J. L. W. V. (1906). Sur les fonctions convexes et les inégalités entre les valeurs moyennes. *Acta Mathematica*, 30(1), 175-193.
- [13] Gini, C. (1912). *Variabilità e mutabilità: contributo allo studio delle distribuzioni e delle relazioni statistiche*. Bologna: C. Cuppini.
- [14] Moran, P. A. P. (1950). Notes on continuous stochastic phenomena. *Biometrika*, 37(1/2), 17-23.

- [15] Hellinger, E. (1909). Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen. *Journal für die reine und angewandte Mathematik*, 136, 210-271.
- [16] Spearman, C. (1904). The proof and measurement of association between two things. *American Journal of Psychology*, 15(1), 72-101.
- [17] Wasserstein, L. N. (1969). Markov processes over denumerable products of spaces describing large systems of automata. *Problems of Information Transmission*, 5(3), 47-52.
- [18] Karhunen, K. (1946). Über lineare Methoden in der Wahrscheinlichkeitsrechnung. *Annales Academiae Scientiarum Fennicae, Series A1: Mathematica-Physica*, 37, 1-79.
- [19] Loève, M. (1946). Fonctions aléatoires de second ordre. *Comptes Rendus de l'Académie des Sciences*, 222, 469-470.
- [20] Lipinski, C. A., Lombardo, F., Dominy, B. W., & Feeney, P. J. (1997). Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 23(1-3), 3-25.
- [21] Veber, D. F., Johnson, S. R., Cheng, H. Y., Smith, B. R., Ward, K. W., & Kopple, K. D. (2002). Molecular properties that influence the oral bioavailability of drug candidates. *Journal of Medicinal Chemistry*, 45(12), 2615-2623.
- [22] Rawat, P., et al. (2023). metapredict: A fast, accurate, and easy-to-use predictor of protein disorder. *Bioinformatics*, 39(2), btad098.
- [23] Erdos, G., et al. (2024). AIUPred: A biophysical model for protein disorder prediction. *Protein Science*, 33(2), e4832.
- [24] Lin, Z., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130.
- [25] Hu, E. J., et al. (2021). LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*.
- [26] Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.